MERN TODO APPLICATION

Full-Stack Web Application

Prepared By: Ritesh Chaudhary

Contents

1. Introduction **2**

2. Project Objectives **2**

3. Technologies and Tools Used **2**

4. System Architecture **3**

4.1 Frontend Layer 3

4.2 Backend Layer 3

4.3 Database Layer 3

4.4 Application Workflow 3

5. Application Features and Implementation **4**

5.1 Todo Creation 4

5.2 Display Todo Records 4

5.3 Search Todo 4

5.4 Edit Todo 4

5.5 Mark Todo as Completed 4

5.6 Delete Todo 4

5.7 Notifications and Error Handling 5

6. Database Design **5**

7. RESTful API Implementation **5**

8. Project Structure **6**

9. Application Demonstration **6**

10. Conclusion **7**

1. Introduction

The MERN Todo Application is a full-stack web application developed to manage everyday tasks efficiently. It provides users with an interface to create, view, search, update, mark as completed, and delete todo items.

The application follows the MERN Stack architecture, integrating MongoDB as the database, Express.js and Node.js for backend development, and React.js for frontend development.

The project demonstrates practical implementation of CRUD (Create, Read, Update, Delete) operations, RESTful API development, database integration, frontend state management, and communication between frontend and backend components.

2. Project Objectives

The main objectives of this project are:

- To develop a functional full-stack todo management application.
- To implement CRUD operations using RESTful APIs.
- To integrate MongoDB with a Node.js and Express.js backend.
- To develop an interactive user interface using React.js.
- To manage application data using React Context API.
- To implement task searching, completion status management, and user notifications.
- To understand the interaction between frontend, backend, and database components.

3. Technologies and Tools Used

Technology / Tool	**Purpose**
React.js	Developing the user interface and reusable components
React Router	Managing frontend navigation and routes
React Context API	Managing and sharing application state
Tailwind CSS	Styling the user interface
React Icons	Providing interface icons
React Hot Toast	Displaying success and error notifications
Node.js	Backend JavaScript runtime
Express.js	Developing RESTful API endpoints
MongoDB	Storing todo records
Mongoose	Database connection and schema modelling
Dotenv	Managing environment variables
Git and GitHub	Source code management and version control
Render	Application deployment

4. System Architecture

The application follows a three-layer architecture consisting of the frontend, backend, and database.

4.1 Frontend Layer

The frontend is developed using React.js. It provides the interface through which users interact with the application.

The frontend consists of reusable components responsible for displaying todo records, creating and editing tasks, searching, deleting, and managing completion status.

React Context API is used to share todo data and application states across components. React Router handles navigation between the home, create, and edit pages.

4.2 Backend Layer

The backend is developed using Node.js and Express.js. It receives HTTP requests from the frontend, processes application logic through controller functions, and communicates with MongoDB using Mongoose.

The backend also includes routing, error handling, JSON request parsing, and static frontend file serving for production deployment.

4.3 Database Layer

MongoDB is used to store todo records. Mongoose defines the Todo schema and provides methods for creating, retrieving, updating, and deleting records.

4.4 Application Workflow

1. The user interacts with the React frontend.
2. The frontend sends an HTTP request to the appropriate backend API endpoint.
3. Express.js receives the request and executes the corresponding controller.
4. The controller communicates with MongoDB through Mongoose.
5. The database returns the requested result.
6. The backend sends a JSON response to the frontend.
7. React updates the interface to display the latest information.

5. Application Features and Implementation

5.1 Todo Creation

Users can create a new todo by entering a title and an optional description. The frontend sends the information to the backend using a POST request.

The backend validates the required title field and saves the new record in MongoDB.

5.2 Display Todo Records

The application retrieves all stored todo records from MongoDB and displays them in a structured table.

The table includes completion status, serial number, title, description, and available actions.

5.3 Search Todo

Users can search for existing tasks using their titles. The frontend filters the retrieved todo records based on the entered search text.

The search operation is case-insensitive.

5.4 Edit Todo

Users can select an existing todo and navigate to its editing page. The application retrieves the selected record using its MongoDB document ID.

After modification, the updated information is submitted to the backend through a PUT request.

5.5 Mark Todo as Completed

The application provides a checkbox for each todo record. Users can change the completion status of a task.

The backend updates the `complete` field in MongoDB, and completed task titles are displayed with a strikethrough effect.

5.6 Delete Todo

Users can delete an existing task using the delete action. Before deletion, a confirmation modal is displayed to prevent accidental removal.

After confirmation, the frontend sends a DELETE request to the backend, which removes the corresponding record from MongoDB.

5.7 Notifications and Error Handling

React Hot Toast provides success and error notifications for user operations.

The application also includes loading indicators, frontend error messages, and centralized backend error-handling middleware.

6. Database Design

The application uses a MongoDB collection named as todos.

The Todo model is defined using Mongoose with the following fields:

Field	**Data Type**	**Description**
_id	ObjectId	Unique identifier generated by MongoDB
title	String	Required and unique task title
description	String	Optional task description
complete	Boolean	Completion status; defaults to false
createdAt	Date	Automatically records creation time
updatedAt	Date	Automatically records the latest update time

Mongoose timestamps are enabled to maintain the creation and modification dates of each todo record.

7. RESTful API Implementation

The backend provides RESTful API endpoints under the base route `/api/todo`.

HTTP Method	API Endpoint	Description
POST	/create-todo	Creates a new todo
GET	/get-all-todos	Retrieves all todo records
GET	/get-todo/:todoId	Retrieves a specific todo
PUT	/edit-todo/:todoId	Updates an existing todo
PUT	/toggleTodo/:todoId	Changes todo completion status
DELETE	/delete-todo/:todoId	Deletes a todo record
GET	/search-todo	Provides a todo search endpoint
GET	/test	Checks API availability

The API communicates using JSON request and response data. Appropriate HTTP methods and status codes are used to represent different operations and their results.

8. Project Structure

The project is organized into separate frontend and backend directories to maintain a clear separation of responsibilities.

Backend responsibilities:

- **controllers/** - Contains application logic for todo operations.
- **models/** - Defines the MongoDB Todo schema.
- **routes/** - Defines API endpoints and connects them to controllers.
- **utils/** - Contains reusable error-handling utilities.
- **index.js** - Configures the Express server, database connection, middleware, and application routes.

Frontend responsibilities:

- **components/** - Contains reusable UI components and application pages.
- **context/** - Manages shared todo data and application state.
- **App.jsx** - Defines application routes and global toast notifications.
- **main.jsx** - Initializes the React application and Context Provider.

9. Application Demonstration

The application provides a straightforward interface for managing todo records.

Users can perform the following operations:

1. View existing todo records on the home page.
2. Search tasks using the search input.
3. Create a new task using the Create New button.
4. Edit an existing task using the edit icon.
5. Mark tasks as completed using checkboxes.
6. Delete tasks using the delete icon and confirmation modal.

Live Application: https://todo-april-2025.onrender.com/

Source Code: https://github.com/riteshchaudhary375/todo-april-2025

10. Conclusion

The MERN Todo Application demonstrates the practical development of a full-stack web application using MongoDB, Express.js, React.js, and Node.js.

Through this project, the implementation of CRUD operations, RESTful API development, database modelling, frontend and backend integration, react component management, and shared state management has been demonstrated.

The project provides practical experience in developing and deploying a database-driven web application while following a structured approach to frontend and backend development.

It also establishes a foundation for further development of full-stack applications involving more advanced functionality, authentication, data management, and application security.